UNIT TEST REPORT

Hospital Management System

1. Introduction

This Unit Test Report documents the frontend automated testing performed on the Hospital Management System. Unit testing verifies that individual application components and their user-facing workflows behave as expected. The current test suite covers the application shell, appointment booking, bed availability, and medicine purchase functionality.

The tests were executed using Vitest and the V8 coverage provider. Component tests use mocked API responses where needed, allowing frontend behaviour to be tested without claiming that backend services or database operations were tested.

2. Objectives of Unit Testing

• To verify that individual frontend components render without crashing.

• To validate appointment booking interactions and request handling.

• To verify bed availability display, ward filtering, allocation, and release workflows.

• To validate medicine listing, selection, quantity limits, cart totals, and order submission.

• To identify failures and measure the extent of executed source code and decision branches.

3. Testing Environment

Component	Description
Application	Hospital Management System – frontend
Programming Language	JavaScript / JSX
Frontend Framework	React 19 with Vite
Testing Framework	Vitest 5.1.0
Coverage Provider	V8 (@vitest/coverage-v8)
DOM Environment	jsdom
Testing Libraries	React Testing Library, jest-dom, user-event
Execution Environment	Windows PowerShell / Visual Studio Code terminal

The frontend tests run in a simulated DOM environment. API calls are mocked within the component tests. This report therefore documents frontend unit/component tests; it does not claim integration testing against a live backend or database.

4. Unit Test Cases

The following table summarizes the 15 tests that passed in the supplied execution output. Test descriptions are grouped by the four test files. The table is a concise summary of the executed suite, not a claim that every possible scenario was tested.

Test ID	Module / Test File	Test Scenario	Expected Result	Actual Result	Status
UT-001	App.jsx	Render application shell	Application renders without crashing	Render test passed	PASS
UT-002	Booking.jsx	Render booking page and doctor list	Booking page and Dr. Smith are displayed	Test passed	PASS
UT-003	Booking.jsx	Display available appointment slots	Available slots are displayed	10:00 and 11:00 slots verified	PASS
UT-004	Booking.jsx	Select a slot and confirm appointment	Appointment POST request is sent and success is shown	Test passed	PASS
UT-005	BedAvailability.jsx	Display bed availability and summary	Bed records and summary counts are shown	Test passed	PASS
UT-006	BedAvailability.jsx	Filter beds by ward	Only beds for selected ward are shown	Test passed	PASS
UT-007	BedAvailability.jsx	Allocate an available bed	Allocation request is processed	Test passed	PASS
UT-008	BedAvailability.jsx	Release an occupied bed	Release request is processed	Test passed	PASS
UT-009	BedAvailability.jsx	Handle bed-loading failure	An error state is displayed	Test passed	PASS
UT-010	MedicinePurchase.jsx	Render medicine list	Medicine list is displayed	Test passed	PASS

Test ID	Module / Test File	Test Scenario	Expected Result	Actual Result	Status
UT-011	MedicinePurchase.jsx	Display empty medicine state	Empty-state message is displayed	Test passed	PASS
UT-012	MedicinePurchase.jsx	Select medicines and calculate cart total	Selected items and total are correct	Test passed	PASS
UT-013	MedicinePurchase.jsx	Enforce quantity maximum	Quantity cannot exceed 20	Test passed	PASS
UT-014	MedicinePurchase.jsx	Create medicine order	Order request is sent and confirmation is shown	Test passed	PASS
UT-015	MedicinePurchase.jsx	Handle medicine-loading failure	An error state is displayed	Test passed	PASS

5. Test Execution Summary

Parameter	Result
Total test files	4
Test files passed	4
Total tests executed	15
Tests passed	15
Tests failed	0
Overall result	PASS
Execution duration	2.68 seconds (as shown in terminal output)

All 15 tests passed successfully across the four test files. No test failures were reported in the supplied terminal execution.

6. Testing Evidence

The following screenshot was supplied as evidence of the test execution. It shows the Vitest coverage command, four passing test files, 15 passing tests, and the V8 coverage summary.

6. Testing Evidence (continued)

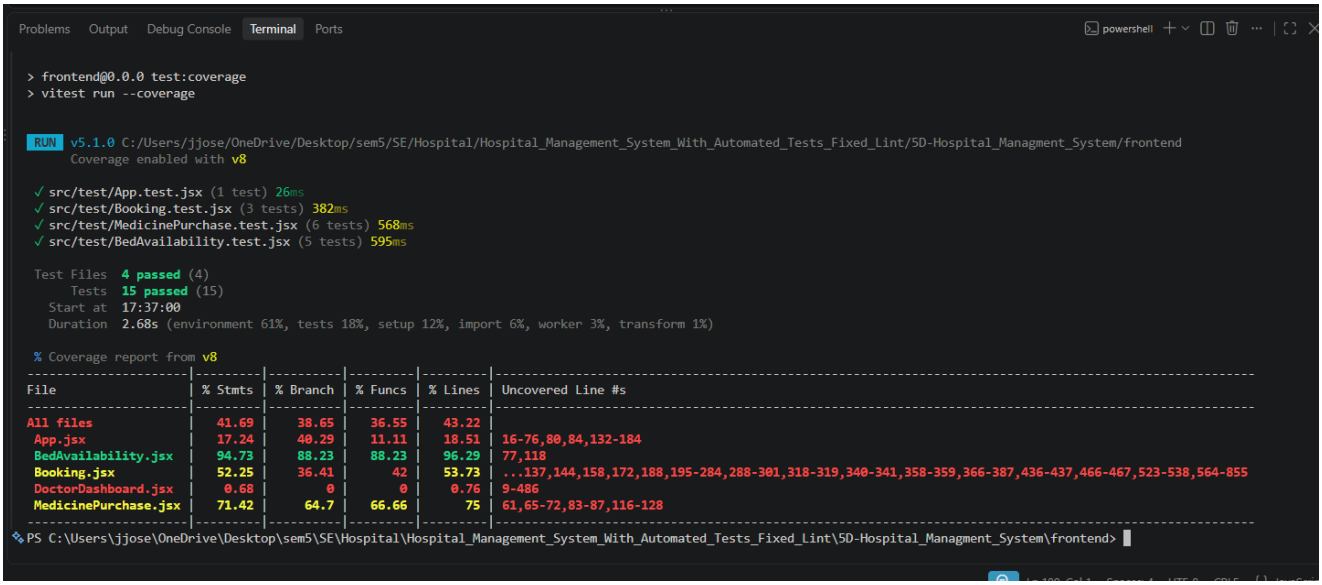


Figure 1. Terminal output showing successful frontend tests and coverage.

7. Conclusion

Unit testing was successfully performed on the frontend of the Hospital Management System using Vitest 5.1.0. The executed suite contained 15 tests across four test files, and all 15 tests passed with zero failures. The tests exercised the application shell, appointment booking, bed availability, and medicine purchase components.

The associated V8 coverage run reported 43.22% line coverage, 38.65% branch coverage, 41.69% statement coverage, and 36.55% function coverage. These figures indicate that the current suite verifies selected frontend behaviour, while additional test cases are needed to exercise the remaining code paths. The results apply to the configured frontend source files and should not be interpreted as backend, database, or complete system test coverage.

The next useful testing priorities are the uncovered paths in Booking.jsx, MedicinePurchase.jsx, and App.jsx, as well as the DoctorDashboard.jsx component, which currently has very low measured coverage.